



Stored Procedures

Introduction to Stored Procedures

Please do not copy without permission. © School of Entrepreneurship and Technology.



What Are Stored Procedures?

Stored procedures are a **collection of SQL statements** that are **stored in the database**, allowing us to **encapsulate** a sequence of operations into a **single callable unit**. Key **characteristics** of stored procedures include:

01. Precompiled

Once created, stored procedures are **compiled by the database server**, which can **improve performance** for repeated operations.

02. Modular

Stored procedures allow us to **organize complex operations** into manageable sections.

03. Reusable

Once a stored procedure is created, it can be **used multiple times** from anywhere in the database, **reducing the need to write** the same SQL code repeatedly.

04. Securable

Permissions can be set on stored procedures, **restricting who can execute them**, which enhances **database security**.



Steps for Creating Stored Procedures

Suppose we're data scientists tasked with analyzing deforestation trends, using data in an SQL database, to aid in the formulation of conservation strategies. We would like to create a stored procedure to calculate how quickly deforestation is occurring:



Step 1: Define the Procedure

This definition sets a clear objective and outlines the inputs needed to achieve your analysis goal.

- **Purpose:** The stored procedure will calculate the rate of deforestation within the specified period and region.
- **Parameters:** Decide on the inputs. For this scenario, parameters might include `start_date`, `end_date`, and `region`.
- **Name:** Choose a clear, descriptive name, such as `Calculate_deforestation_rate`.



Step 2: Craft the SQL Statements

Write the SQL statements that form the core of our stored procedure.

- **Data Retrieval:** Use `SELECT` statements to gather historical deforestation data from the databases.
- **Calculations:** Implement SQL logic to compute the rate of deforestation based on the difference in forest cover between the start and end dates.
- **Result Format:** Structure the output to provide clear insights, such as the percentage decrease in forest area.



Steps for Creating Stored Procedures



Step 3: Enhance Performance and Efficiency

Optimization ensures that your analysis can scale and adapt to growing datasets, maintaining responsiveness and accuracy.

- **Indexing:** Apply indexing on columns involved in filtering and joining, such as **Region** or **Start_date**, to speed up data retrieval.
- **Avoiding Cursors:** Cursors are tools used to retrieve, manipulate, and navigate through the rows of a result set one row at a time. They can be slow and resource-intensive. Where possible, use set-based operations to manipulate data more efficiently.



Step 4: Test and Debug

This critical phase validates the functionality and effectiveness of our stored procedure, ensuring it's ready for real-world application.

- **Create Test Data:** Use a subset of real-world data that reflects various scenarios.
- **Validation:** Check if the calculated deforestation rates from the test data align with known benchmarks or external datasets.
- **Adjustments:** Refine the SQL statements based on the test outcomes, ensuring accuracy and reliability.



Step 5: Share Findings

Our work can now be shared with researchers and policymakers to enable them to :

- **Understand Trends:** Quickly assess deforestation rates in various regions.
- **Make Informed Decisions:** Base conservation strategies on accurate, up-to-date data.
- **Monitor Progress:** Measure the effectiveness of conservation initiatives over time.



Parameters in Stored Procedures

Parameters are **variables that are declared in the stored procedure's definition**. They can be used to **pass data into and out** of stored procedures. There are three types of parameters:

IN: The default type. This allows you to **pass a value** into the stored procedure.

OUT: This allows the stored procedure to **return a value** to the caller.

INOUT: This allows **both input** from the caller and **output** from the stored procedure.

Basic example of how to create a stored procedure in MySQL Workbench:

```
CREATE PROCEDURE ProcedureName(IN InputParam INT, OUT OutputParam INT)  
BEGIN  
    -- SQL statements here  
END
```

To call this stored procedure:

```
CALL ProcedureName(@InputParam, @OutputParam);
```



Advantages of Using Stored Procedures

01. Performance Improvement

Stored procedures are precompiled, which means the database engine can execute them **faster than dynamic SQL queries**, which need to be compiled and optimized each time they are run.

02. Reduced Network Traffic

Encapsulating operations within a stored procedure **minimizes the amount of data** sent over the network because only calls to the procedure and its parameters—rather than lengthy SQL queries—need to be transmitted.

03. Enhanced Security

Stored procedures provide a **security layer by restricting direct access to the data**. They allow users to perform specific operations without giving them direct access to the database tables, thus protecting sensitive data.

Consistency and Integrity

They ensure **operational consistency** and **data integrity** by executing the same SQL commands in the **same order**, thereby avoiding discrepancies and ensuring reliable results.

Easier Maintenance

Any changes to the query logic can be made within the stored procedure without affecting the applications that call it. This **centralizes the logic** and makes **maintenance more straightforward**.

Parameterization

Stored procedures allow parameterization, making it **easier to pass input** values and use them in SQL queries. This feature also helps to **prevent SQL injection attacks** by clearly differentiating between the code and the data.

04.

05.

06.



Advantages of using stored procedures

07. Transactional Control

Encapsulation also ensures that **all** operations either **complete successfully together** or are **all rolled back**, maintaining database **integrity**.

08. Better Resource Allocation

Because the database server handles the execution of stored procedures, they can be **optimized to utilize server resources** more efficiently, enhancing overall database **performance**.

09. Customizable Execution Logic

Stored procedures can contain **complex logic**, such as loops and conditional statements, allowing for more **sophisticated** data manipulation and business logic implementation directly within the database.

10. Simplification of Complex Operations

They can simplify complex operations by **hiding the complexity from the user**. Complex SQL queries with joins, subqueries, and conditional logic can be **encapsulated** within stored procedures, making them easier to execute.

11. Audit and Logging Capabilities

Stored procedures can be designed to include auditing and logging functionality, **recording changes and access to data**, which is crucial for **compliance** and **security monitoring**.

12. Reusability Across Applications

Once defined, stored procedures can be **called from various applications and services**, promoting **code reuse** and **consistency** across different projects and platforms.



Best Practices for Using Stored Procedures

01. Modular Design

Breaks down the data analysis into **smaller stored procedures**, each focusing on a **specific aspect**, such as forest cover or species diversity. This approach **simplifies development** and **enhances clarity**.

02. Error Handling

Implement error handling within the stored procedures to ensure the **analysis can proceed smoothly**, even with imperfect data, by managing exceptions gracefully.

03. Documentation

Document each stored procedure's **purpose**, **parameters**, and **usage**, making it easier for team members to understand and collaborate on the project.

04. Security Considerations

Prioritize security in stored procedures to prevent **vulnerabilities**, such as SQL injection, ensuring **data integrity** and **privacy**.